

Web 开发技术 @ 2025

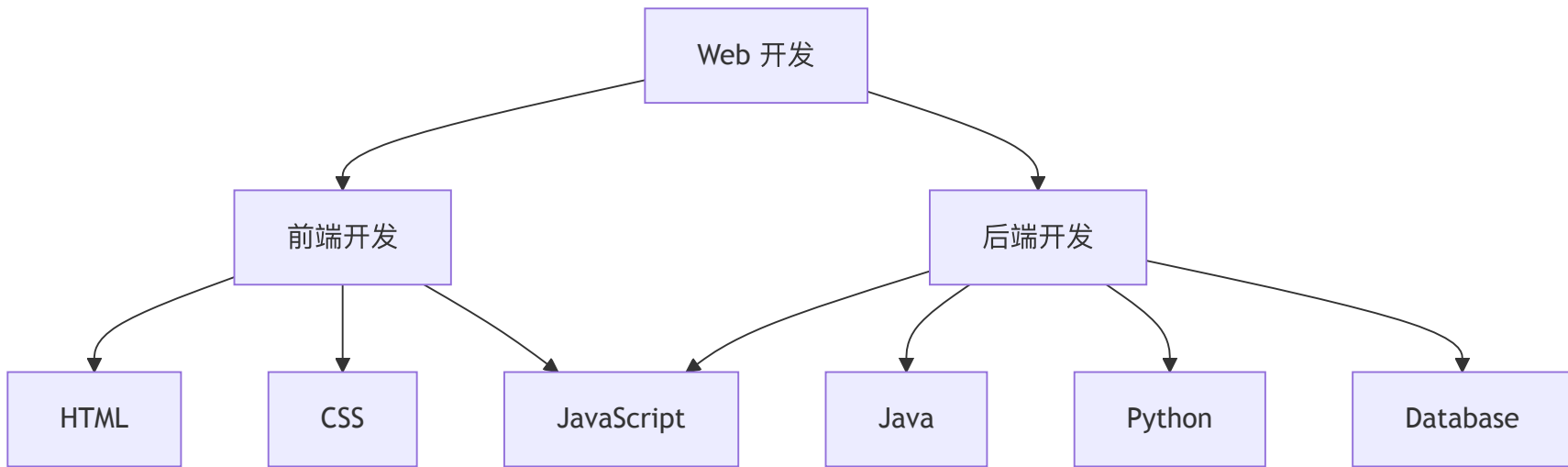
章许帆

xufan.zhang@outlook.com

Web 开发基础

The background image is a high-contrast, atmospheric photograph of a mountain landscape. A prominent, jagged mountain peak rises from a valley, its slopes covered in brownish, rocky terrain. A narrow, winding path leads up the mountain, with a small cluster of buildings or a camp visible near the base. The scene is partially shrouded in mist and low-hanging clouds, creating a sense of mystery and scale. The sky is filled with dark, heavy clouds, adding to the dramatic and somewhat somber mood of the image.

Web 开发技术概览 (部分)



SGML

Standard Generalized Markup Language，是一种标准通用标记语言

SGML提供了一种描述对象、元素和属性之间关系的方法，并告诉计算机如何识别文档的组成部分。它提供了计算机识别文本实体的各种元素开始和结束位置的规则。

SGML 结构

```
<anthology>
  <poem>
    <title>全栈吟</title>
    <stanza>
      <line>屏前指落键声轻,</line>
      <line>指下千行码自生。</line>
      <line>莫笑前端雕玉苦,</line>
      <line>一次刷新万窗晴。</line>
    </stanza>
  </poem>
  <!-- 我是注释 -->
</anthology>
```

HTML

超文本标记语言是一种用来结构化 Web 网页及其内容的标记语言。

职责: 定义网页内容的含义与结构

```
<html>
  <head></head>
  <body>
    <header>
      <title>学码偶得</title>
    </header>
    <ul>
      <li>夜深独对屏幕光, </li>
      <li>手拙频翻文档长。</li>
      <li>忽见终端报捷讯, </li>
      <li>你好世界映朝阳。</li>
    </ul>
  </body>
</html>
```

HTML cheatsheet: <https://developer.mozilla.org/en-US/docs/Web/HTML/Guides/Cheatsheet>

HTML 元素（部分）

HTML 由一系列的元素组成，这些元素可以用来包围或标记不同部分的内容，使其以某种方式呈现或者工作。

- **html.** 表示一个 HTML 文档的根，所有其它元素都必须是此元素的后代。
- **head.** 包含文档相关的配置信息（元数据），包括文档的标题、脚本和样式表等。
 - **link.** 指定当前文档与外部资源的关系。该元素最常用于链接 CSS，也可以用来创建站点图标。
- **body.** 表示文档的内容，一个文档中只能有一个该元素。
 - **div, ol, ul, p.** 用于组织文档内容部分的内容结构。
 - **form, input, button.** 用于组织文档内容部分的表单。
- **script.** 用于嵌入可执行脚本或数据。通常用作嵌入或者引用 JavaScript 代码。
- **canvas.** 用来通过 canvas scripting API 或 WebGL API 绘制图形及图形动画的容器元素。
- **audio, img, video.** 用于在文档中嵌入音频、图像、视频内容。

<https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements>

HTML 元素嵌套规则

HTML 元素支持满足一定规则的嵌套

■ 块级元素 (Block elements)

块级元素总是从新行开始，总是占据整个可用宽度（尽可能地向左和向右延伸）。浏览器会自动在元素前后添加一些空格（边距）。

```
<div> <p> <article> <canvas> <ol> <ul> <form> ...
```

■ 行内元素 (Inline elements)

内联元素不会从新行开始，只占用必要的宽度。

```
<a> <sub> <sup> <button> <label> <span> ...
```

规则：

块级元素可以包含某些块元素或者行内元素；行内元素只能包含行内元素，不能包含块级元素

HTML 元素属性

所有的HTML 元素都可以拥有属性，属性提供关于元素额外的信息

属性始终在元素的开始标签中指定，并以名称/值对的形式出现，形如：name="value"

- <a> 标签的 href 属性制定链接指向的 URL

```
<a href="https://software.nju.edu.cn">学院官网</a>
```

- 标签的 src 属性指定图片的路径

```

```

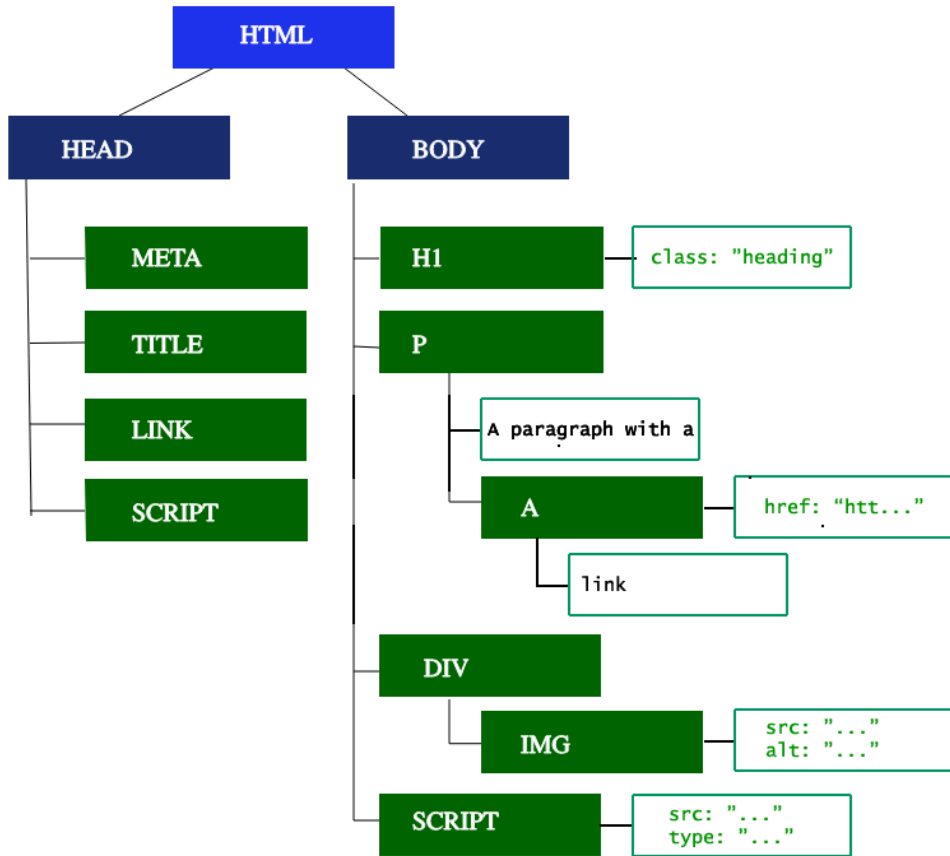
- <link> 标签的 rel 属性用于设置对象和链接目的间的关系。

```
<link rel="stylesheet" href="index.css" />
```

当 rel 的值为 stylesheet 时，表示链接的是一个外部加载的样式表。

开发者也可以为元素自定义属性，实现灵活的数据存储和交互机制。

HTML 树状结构 (DOM)



练习

实现: **This is** _{subscript} **and** ^{superscript}

HTML Code Runner

RESET

```
This is subscript and superscript
```

输出

This is subscript and superscript

CSS

层叠样式表 (Cascading Style Sheets) 描述 HTML 文档的呈现方式

- HTML 元素 `style` 属性

HTML Code Runner

RESET

```
<div>
  <h2>习HTML作</h2>
  <div>
    <div>标签如骨立框架, </div>
    <div>属性精调塑物华。</div>
    <div>万页千端皆有序, </div>
    <div>一屏世界码为家。</div>
  </div>
</div>
```

输出

习HTML作

标签如骨立框架，
属性精调塑物华。
万页千端皆有序，
一屏世界码为家。

CSS

CSS 基本目标是让浏览器以指定的特性去绘制页面元素

CSS 语法由两部分构建，核心功能是将 CSS 属性设定为特定的值：

- 属性：一个标识符，用可读的名称来表示其特性
- 值：描述了浏览器引擎如何处理该特性，每个属性都包含一个有效值的集合

CSS 引擎会计算页面上每个元素都有哪些属性与值的键值对，并且会根据结果绘制元素，排布样式。

CSS 规则 = 选择器 + 声明块

```
poem-container {  
  background-color: gray;  
  border-radius: 5px;  
}
```

选择器类型及优先级

一个元素可能被多个选择器选中，浏览器通过优先级来判断哪些属性值与一个元素最为相关，从而在该元素上应用这些属性值。

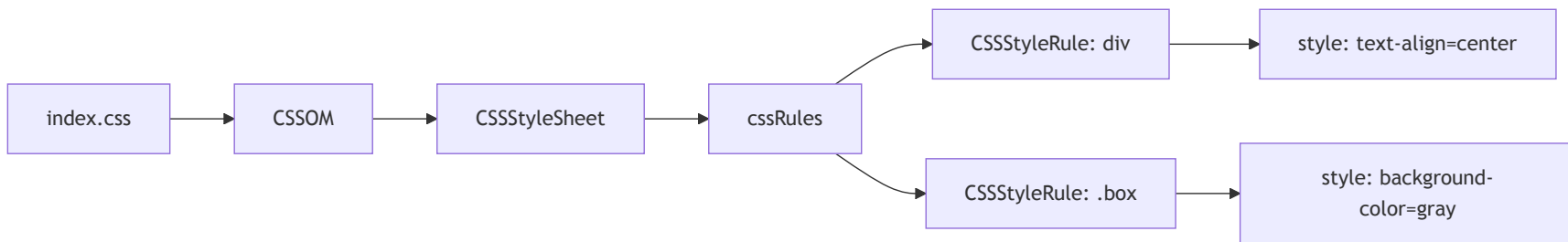
选择器类型	优先级
类型选择器 (div)，伪元素 (::before)	低
类选择器 (.poem-line)，属性选择器 (type="text")，伪类	中
ID选择器 (#root)	高

元素的内联样式（例如，`style="font-weight:bold"`）总会覆盖外部样式表的任何样式，具有最高的优先级。

CSSOM

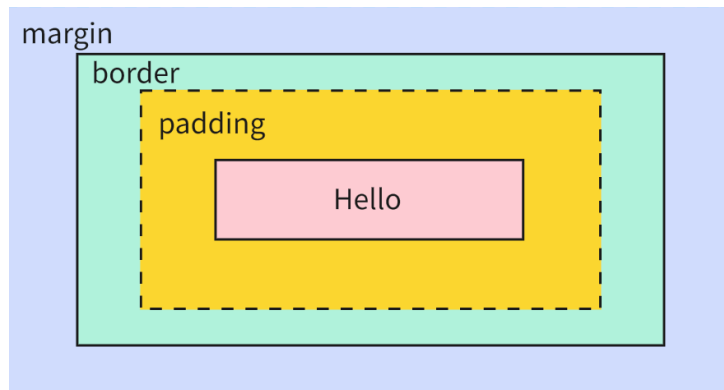
浏览器将 CSS 代码转换为可编程对象模型的代表方式

```
/* index.css */
div {
  text-align: center;
}
.poem-container {
  background-color: gray
}
```



CSS 盒模型

所有的HTML元素都可以表示成一个个矩形盒子，CSS 决定这些盒子的大小、位置等属性



- 内容区域由内容边界限制，容纳着元素的真实内容，例如文本、图像，或是一个视频播放器。
- 内边距区域由内边距边界限制，扩展自内容区域，负责延伸内容区域的背景，填充元素中内容与边框的间距
- 边框区域由边框边界限制，扩展自内边距区域，是容纳边框的区域。
- 外边距区域由外边距边界限制，用空白区域扩展边框区域，以分开相邻的元素。

JavaScript 语法

JavaScript 描述网页内容的功能与行为

- 变（常）量声明：先声明，后使用

```
var foo = 'bar';  
let goo = 'bar';  
const hoo = 'bar';
```

JavaScript Code Runner

RUN

RESET

输出

```
var foo = "bar";  
console.log(foo);
```


JavaScript 语法

JavaScript 是弱类型语言，变量没有类型，而值有类型

变量命名规则: 以字母(a-zA-Z)、下划线 (_) 或美元符号 (\$) 开头，后续字符也可以是数字 (0-9)。

值类型:

- Boolean. true, false
- Number. 整数或者浮点数, e.g., 1, 3.14
- String. 一串字符序列, "Hello"
- null. 空值
- undefined. 未初始化的变量或者不存在的属性
- BigInt. 任意精度的整数, 9841315219931017n
- Symbol. 实例是唯一且不可变的一种数据类型, Symbol("foo") === Symbol("foo")
- Object. 可以看做是一组属性的集合, {name: 'Xufan Zhang', email: 'xufan.zhang@outlook.com'}

JavaScript 集合

Map, Set, Array

- Array: `shift`, `pop`, `push`, `concat`, `splice`, `slice`, ...
- Map: `set`, `get`, `has`, `delete`, ...
- Set: `add`, `has`, `delete`, ...

JavaScript Code Runner	RUN	RESET	输出

JavaScript Class

类是创建对象的模板。JavaScript中的类建立在原型之上，实现属性和操作方法的定义

```
class Poem {  
  constructor(title, content) {  
    this.title = title;  
    this.content = content;  
  }  
  
  public print() {  
    let title = "<h2>" + this.title + "</h2>"  
    let content = "<div>";  
    for(const item of this.content) {  
      content += "<div>" + item + "</div>"  
    }  
    content += "</div>"  
    return "<div>" + title + content + "</div>"  
  }  
}
```

JavaScript 函数

函数是第一等公民

$y = f(x)$ ， y - 输出， x - 输入

```
function name(param) {  
    ...  
}
```

一个完整的 JavaScript 函数包括函数名、参数列表和函数体。与C、Java等强类型语言相比，有何不同？

函数表达式：

```
const name = function(param) {  
    ...  
}
```

```
const name = (param) => {  
    ...  
}
```

JavaScript Closure

闭包是指有权访问另一个函数作用域中变量的函数，即使外部函数已经执行完毕。

JavaScript Code Runner	RUN	RESET	输出
<pre>function outer() { const outerVar = '我在外部函数中'; function inner() { console.log(outerVar); // 访问外部函数变量 } return inner; } const closureFunc = outer(); // outer执行完毕 closureFunc();</pre>			

形成条件: 函数嵌套，内部函数引用了外部函数的变量，且内部函数在外部函数之外被调用。

JavaScript 控制流

- 块语句 (Block Statement)

```
{  
    ...  
}
```

- 条件语句 (Conditional Statement)

```
if(condition) {  
    ...  
} else {  
    ...  
}
```

```
switch(value) {  
    case '':  
        ...  
        break;  
    ...  
    default:  
        ...  
}
```

JavaScript 循环

```
for(initialiazation; condition; afterthought) {  
    ...  
}
```

1. 执行初始化表达式初始化，通常初始化一个或多个循环计数器。
2. 计算条件表达式。如果condition的值为true，则执行循环语句；否则，循环终止。
3. 块语句执行。
4. 执行事后更新表达式。
5. 控制返回到步骤2。

```
while(condition) {  
    ...  
}
```

```
do {  
    ...  
} while(condition)
```

JavaScript Promise

异步操作，基于AJAX的请求都是异步执行的

Promise 的三种状态:

- 等待中 (pending), 初始状态, 既不是成功也不是失败
- 成功 (fulfilled), 操作成功
- 失败 (rejected), 操作失败

```
const promise = new Promise((resolve, reject) => {  
  ...  
});  
promise.then(value => {  
  ...  
})  
promise.catch(err => {  
  ...  
})  
promise.finally(() => {  
  ...  
})
```


JavaScript 运算符

■ 赋值运算符

=, +=, -=, *=, /=, %=, &=, ^=, |=, ...

■ 比较运算符

>, <, >=, <=, ==, ===, !=, !==

JavaScript Code Runner	RUN	RESET	输出
<pre>console.log(1 == 1); console.log('hello' == 'hello'); console.log('1' == 1); console.log(0 == false);</pre>			

■ 算数运算符

+, -, *, /, %, ++, --, **

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Expressions_and_operators

练习

斐波那契数（通常用 $F(n)$ 表示）形成的序列称为 斐波那契数列 。该数列由 0 和 1 开始，后面的每一项数字都是前面两项数字的和。

$F(0) = 0, F(1) = 1$

$F(n) = F(n - 1) + F(n - 2)$, 其中 $n > 1$

给定 n ，请计算 $F(n)$ 。

JavaScript Code Runner

RUN

RESET

输出

```
/**
 * @param {number} n
 * @return {number}
 */
const fib = function(n) {
};
```

JavaScript 操作 DOM

■ 增

```
document.createElement('div'); // 创建元素
document.createTextNode('你好世界'); // 创建文本节点
parentElement.appendChild(newElement); // 追加到子节点末尾
```

■ 删

```
parentElement.removeChild(childElement); // 删除节点
childElement.remove();
```

■ 改

```
parentElement.replaceChild(newElement, oldElement); // 替换节点
element.innerHTML = '<strong>新内容</strong>'; // 修改HTML内容
inputElement.value = '新值'; // 获取/设置表单元素值
```

■ 查

```
document.getElementById('header'); // 通过ID获取
document.querySelector('.menu-item'); // 通过CSS选择器获取
document.getElementsByClassName('item'); // 通过类名获取
```

JavaScript 操作 CSSOM

读取和修改 CSS 样式

```
const styleSheets = document.styleSheets; // 返回StyleSheetList

const firstStyleSheet = document.styleSheets[0]; // 获取第一个样式表
```

■ 读取样式

```
const bgColor = styleSheet.cssRules[0].style.backgroundColor; // 获取第一条规则的背景色

const priority = styleSheet.cssRules[0].style.getPropertyPriority('color'); // 获取特定属性的优先级
```

■ 修改样式

```
styleSheet.cssRules[0].style.color = 'red'; // 修改样式属性

styleSheet.cssRules[0].style.setProperty('background-color', 'blue', 'important'); // 使用setProperty方法

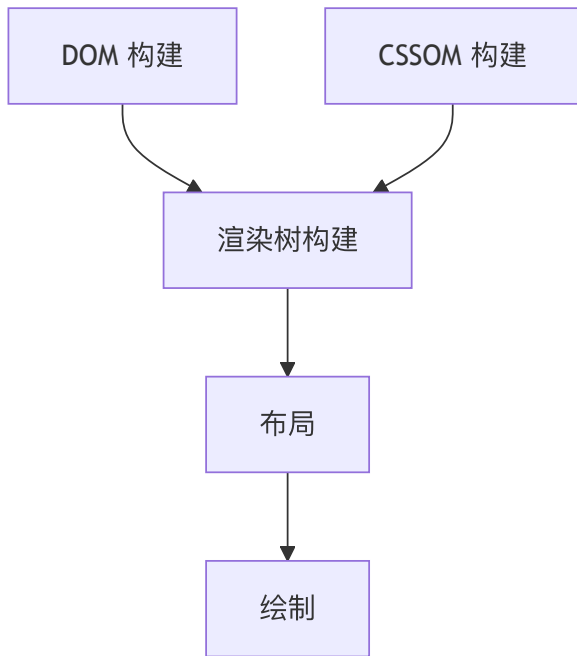
styleSheet.insertRule('body { background: #fff; }', 0);

styleSheet.deleteRule(0);
```

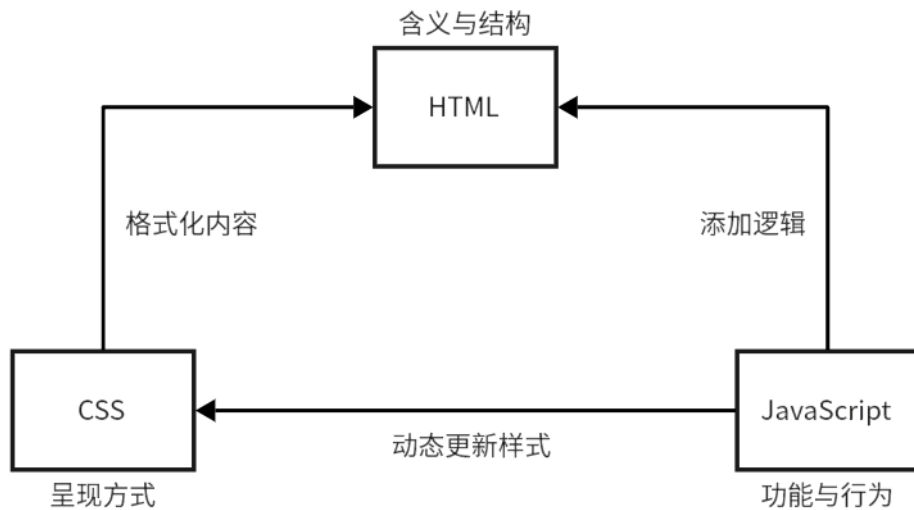
Web 关键渲染路径 (Critical Rendering Path)

- **DOM 构建:** 包含页面所有的内容。DOM 树由 DOM 节点构成。单个 DOM 节点以起始标签标记开始，以结束标签标记结束。节点包含有关 HTML 元素所有相关信息。节点根据标记的层次结构连接到 DOM 树中。
- **CSSOM 构建:** 包含页面所有的样式。CSSOM 的构建是全量的，直到所有的 CSS 文件被接收和执行，才能够继续构建渲染树。样式的规则越具体，耗时越高，但这个耗时通常是毫秒级，通常不值得优化。
- **渲染树构建:** 包含内容和样式，将 DOM 和 CSSOM 树结合为渲染树。浏览器从 DOM 树的根节点开始，决定附加哪些 CSS 规则，渲染树只包含可见内容，直至所有节点遍历完成。
- **布局:** 布局取决于屏幕的尺寸。布局步骤决定了元素在页面上的位置和布局方式，包括宽、高、位置关系等。当设备旋转或者浏览器缩放时，会重新进行布局。布局性能受到 DOM 的影响。
- **绘制:** 一旦渲染树创建并且布局完成，像素就可以被绘制在屏幕上。加载时，整个屏幕被绘制出来。之后，只有受影响的屏幕区域会被重绘，浏览器被优化为只重绘需要绘制的最小区域。

Web 关键渲染路径 (Critical Rendering Path)



HTML CSS JavaScript



A dramatic landscape photograph of a mountain range. A sharp, rocky peak rises from a valley, partially shrouded in mist. The foreground shows a steep, brownish slope with a winding path and a small cluster of buildings. The sky is filled with heavy, dark clouds.

谢谢